Mini juego «Piedra, Papel o Tijera» con persistencia de datos (struct + fstream)

Alexis Maier

E.E.S.T. N.º 1 «Eduardo Ader», Vicente López

Laboratorio de Programación (LPR) — 5.º año 3.ª división, grupos A-B

Profesor: Mansilla Muñoz York Elías

21 de octubre de 2026

v1.0.0 — Baseline aprobado

Mini juego «Piedra, Papel o Tijera» con persistencia de datos (struct + fstream)

Introducción

La memoria RAM es volátil: sus datos se pierden al cerrar el programa. El almacenamiento secundario (disco) es no volátil y permite conservar la información en archivos. Esta actividad desarrolla un juego de Piedra, Papel o Tijera contra la computadora en el que cada partida se modela con un `struct` y se guarda en un archivo de texto, `historial_partidas.txt`.

Se emplea `ofstream` en modo *append* (`ios::app`) para agregar partidas sin sobrescribir las anteriores e `ifstream` para recuperar el historial. Integrante: Alexis Maier (trabajo individual).

Desarrollo

Diseño

El `struct Partida` agrupa `idPartida`, `jugador`, `eleccionJugador`, `eleccionIA` y `resultado`. El menú, dentro de un `do...while`, permite jugar (opción 1), ver el historial (opción 2) o salir (opción 3). `jugarPartida` usa `cin.ignore()` antes de `getline` para limpiar el búfer, genera la jugada de la IA con `rand()` y evalúa el resultado. `guardarEnDisco` escribe una línea CSV por partida y `leerHistorialDisco` la lee con `getline` delimitado por comas.

Compilación y demostración de persistencia

Se compiló con `g++ -std=c++11 src/main.cpp -o src/piedrapapelotijera.exe`. En la primera sesión se jugaron dos partidas y se consultó el historial; luego se cerró el programa, se volvió a abrir y la opción 2 mostró las mismas partidas leídas desde el disco.

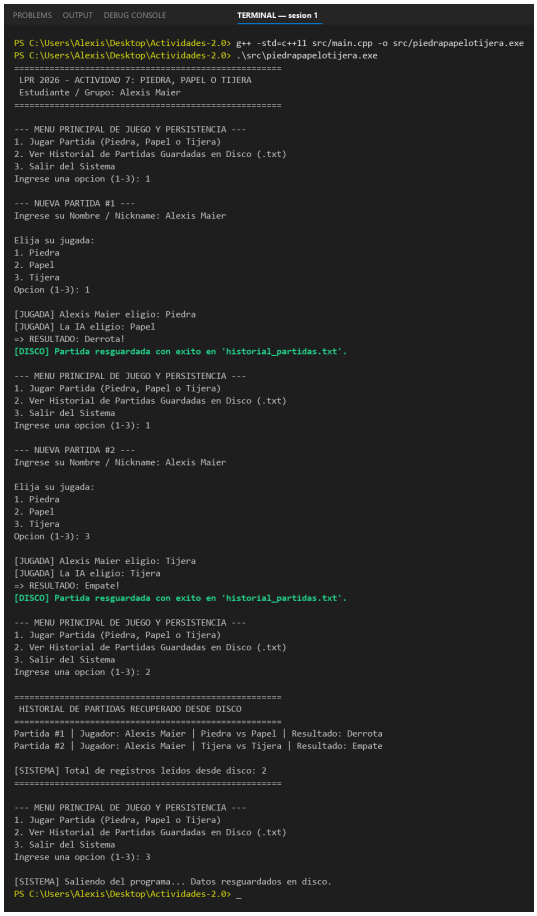


Figura 1. Sesión 1: menú, dos partidas jugadas, historial y salida.

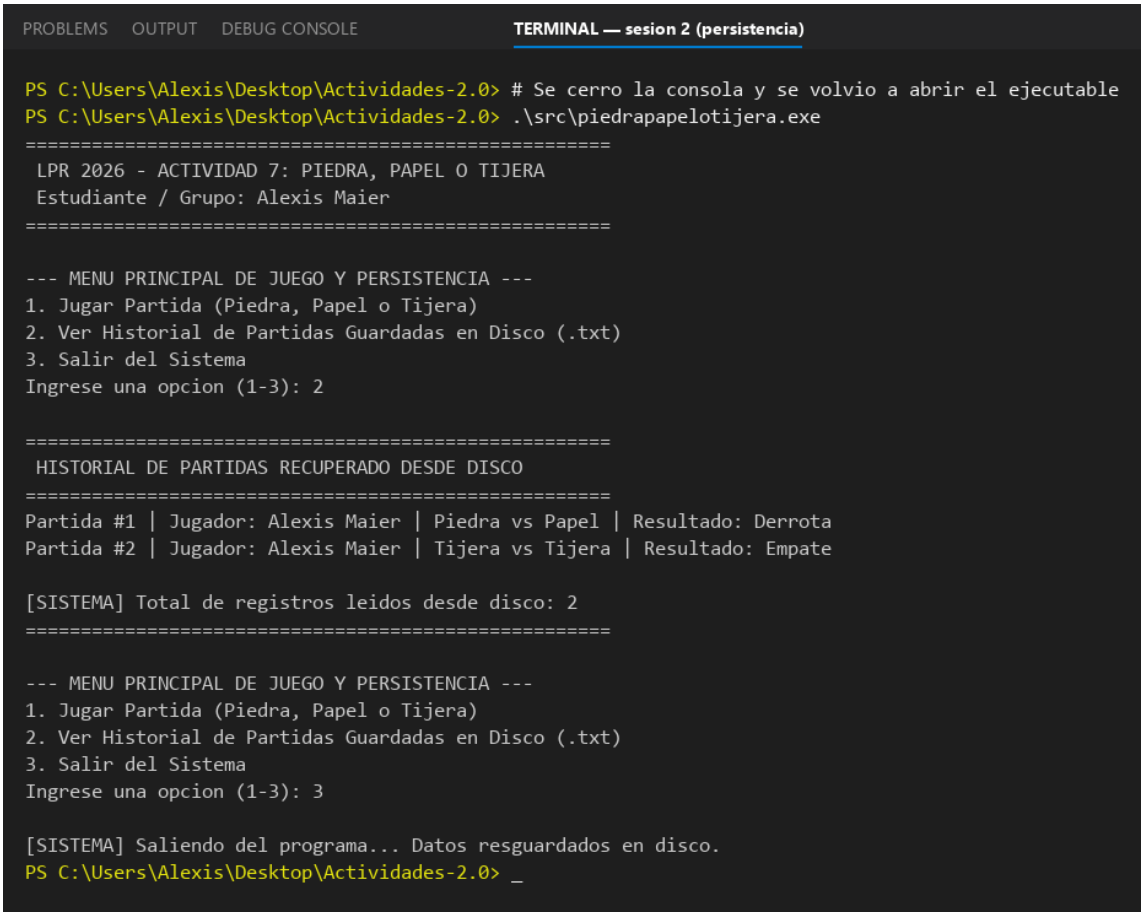


Figura 2. Sesión 2: el programa reabierto recupera el historial desde el disco.

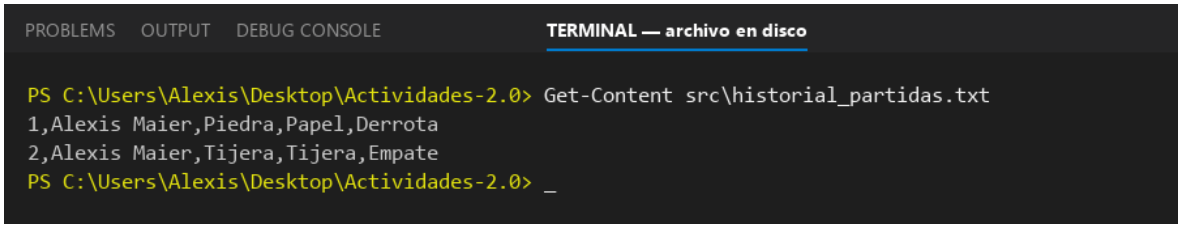


Figura 3. Contenido de src/historial_partidas.txt en formato CSV.

Cuestionario técnico de control

1. RAM vs. Disco: ¿Por qué si cerrás la ventana de consola y volvés a abrir el ejecutable, la Opción 2 sigue mostrando el historial de partidas anteriores?

Porque las partidas no se guardan solo en variables (RAM, volátil), sino que `guardarEnDisco` las escribe en `src/historial_partidas.txt`, que está en el almacenamiento secundario y no se borra al cerrar el programa. Al volver a abrirlo, la opción 2 lee ese archivo con `ifstream` y recupera todo. Se demostró en la Figura 2.

2. Uso de ios::app: ¿Qué sucedería con el archivo si en guardarEnDisco utilizaras ofstream archivo("historial_partidas.txt"); sin incluir ios::app?

El modo por defecto de `ofstream` es `ios::out`, que trunca (vacía) el archivo al abrirlo. Cada partida nueva borraría las anteriores y el historial siempre tendría una sola línea, la última.

3. Manejo del búfer (cin.ignore()): ¿Por qué es necesario invocar cin.ignore() antes de leer el nombre con getline(cin, p.jugador)?

Después de leer la opción del menú con `cin >> opcion`, el Enter queda pendiente en el búfer. `getline` lo toma enseguida como una línea vacía y no espera al usuario, por lo que el nombre quedaría en blanco. `cin.ignore()` descarta ese salto de línea.

4. Agrupación con struct: ¿Qué ventajas ofrece la estructura Partida frente a pasar 5 variables sueltas a cada función?

Agrupa los datos relacionados en una sola entidad: `guardarEnDisco(const Partida& p)` recibe un único parámetro, por referencia constante (sin copiar ni modificar), el código es más legible, y agregar un campo nuevo solo exige cambiar el struct y no las firmas de todas las funciones.

5. Aplicación al P.I.A.: Si adaptaran este módulo de almacenamiento a su Proyecto Integrador Anual, ¿qué datos o eventos de su prototipo guardarían en el archivo de texto?

Se guardarían registros de eventos del prototipo como lecturas de sensores (fecha y hora, identificador del sensor, valor medido) o un registro de usuarios y de alarmas. Bastaría cambiar los campos del struct `Partida` por los de la lectura y mantener el mismo formato CSV, que también se puede abrir desde una planilla de cálculo.

Conclusión

La actividad permitió distinguir la RAM del almacenamiento en disco y comprobar que la persistencia depende de escribir en un archivo. El `struct` ordena los datos, el modo `ios::app` evita perder el historial y `cin.ignore()` resuelve el problema típico del búfer de entrada. Se identificaron limitaciones (los ID se reinician en cada ejecución y los nombres con comas romperían el CSV) que quedaron documentadas en el manual del programador.

Referencias

cppreference.com. (s. f.). *std::ios_base::openmode*.
https://en.cppreference.com/w/cpp/io/ios_base/openmode

Microsoft. (s. f.). *Flujos de archivos en C++*.
https://learn.microsoft.com/es-es/cpp/standard-library/

GeeksforGeeks. (s. f.). *Structures in C++*.
https://www.geeksforgeeks.org/structures-in-cpp/

Mansilla Muñoz, Y. E. (2026). *Actividad 7: Mini juego «Piedra, Papel o Tijera» con persistencia de datos* [Guía de actividad]. E.E.S.T. N.º 1 «Eduardo Ader».